COMSATS UNIVERSITY ISLAMABAD

(ABBOTTABAD CAMPUS)

LAB TASK – 4

MAIN TOPIC: **JUNIT TESTING**

COURSE: **SOFTWARE QUALITY ENGINEERING**

COURSE TEACHER: **SIR. MUKHTIAR ZAMIN**

SUBMITTED BY: **MUHAMMAD KHURRAM SHEHZAD**

REGISTRATION NO.: **FA23-BSE-070**

CLASS: **BSE-6B**

SUBMITTED ON: **14TH MAY, 2026**

Table of contents

QUESTION – 1:.....**3**

SOLUTION:.....**3**

CODE:.....3
TEST CLASS:.....3
OUTPUT:.....4

QUESTION – 2:.....**5**

SOLUTION:.....**5**

CODE:.....5
TEST CLASS:.....5
OUTPUT:.....6

QUESTION – 3:.....**7**

SOLUTION:.....**7**

CODE:.....7
TEST CLASS:.....8
OUTPUT:.....9

QUESTION – 1:

WRITE A PROGRAM IN JAVA TO SOLVE THE TRIANGLE PROBLEM DEFINED AS FOLLOWS:

ACCEPT THREE INTEGERS WHICH ARE SUPPOSED TO BE THE THREE SIDES OF A TRIANGLE AND DETERMINE IF THE THREE VALUES REPRESENT AN EQUILATERAL TRIANGLE, ISOSCELES TRIANGLE, SCALENE TRIANGLE, OR THEY DO NOT FORM A TRIANGLE AT ALL.

WRITE UNIT TEST CASES FOR THE TRIANGLE CLASS. WHICH WILL CHECK THE FUNCTIONALITY OF TRIANGLE CLASS.

SOLUTION:

CODE:

```
public class Triangle {

 public String determineType(int a, int b, int c) {

 if (a <= 0 || b <= 0 || c <= 0) {
 return "Invalid";
 }

 if (a + b <= c || a + c <= b || b + c <= a) {
 return "Invalid";
 }

 if (a == b && b == c) {
 return "Equilateral";
 } else if (a == b || b == c || a == c) {
 return "Isosceles";
 } else {
 return "Scalene";
 }
 }
}
```

TEST CLASS:

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class TriangleTest {

    Triangle triangle = new Triangle();

    @Test
    void testEquilateralTriangle() {
```

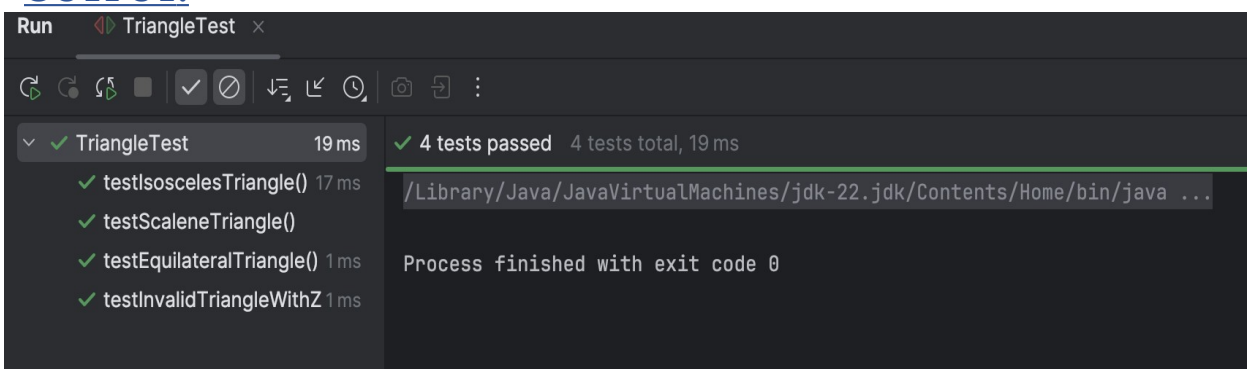
```
        assertEquals("Equilateral", triangle.determineType(5, 5, 5));
    }

    @Test
    void testIsoscelesTriangle() {
        assertEquals("Isosceles", triangle.determineType(5, 5, 4), "Fails
when a and b are not equal");
        assertEquals("Isosceles", triangle.determineType(5, 3, 5), "Fails
when a and c are not equal");
    }

    @Test
    void testScaleneTriangle() {
        assertEquals("Scalene", triangle.determineType(7, 9, 12));
    }

    @Test
    void testInvalidTriangleWithZeroOrNegativeSides() {
        assertEquals("Invalid", triangle.determineType(0, 5, 5), "Fails on 0
side");
        assertEquals("Invalid", triangle.determineType(5, -1, 5), "Fails on
negative side");
    }
}
```

OUTPUT:



QUESTION – 2:

WRITE CALCULATOR CLASS HAVING METHODS ADDITION, MULTIPLICATION , SUBTRACT, SQUARE, CUBE. WRITE TEST CASES TO TEST THE FUNCTIONALITY OF CALCULATOR CLASS.

SOLUTION:

CODE:

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }

    public int square(int a) {
        return a * a;
    }

    public int cube(int a) {
        return a * a * a;
    }
}
```

TEST CLASS:

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {
    Calculator calc = new Calculator();

    @Test
    void testAddition() {
        assertEquals(10, calc.add(5, 5), "Standard addition failed");
        assertEquals(0, calc.add(5, -5), "Adding negative number failed");
        assertEquals(-10, calc.add(-5, -5), "Adding two negative numbers
failed");
    }

    @Test
    void testSubtraction() {
        assertEquals(5, calc.subtract(10, 5), "Standard subtraction failed");
        assertEquals(15, calc.subtract(10, -5), "Subtracting a negative
```

```
number failed");
        assertEquals(0, calc.subtract(-5, -5), "Subtracting two negative
numbers failed");
    }

    @Test
    void testMultiplication() {
        assertEquals(25, calc.multiply(5, 5), "Standard multiplication
failed");
        assertEquals(-25, calc.multiply(5, -5), "Multiplying by a negative
failed");
        assertEquals(0, calc.multiply(5, 0), "Multiplying by zero failed");
        assertEquals(25, calc.multiply(-5, -5), "Multiplying two negatives
failed");
    }

    @Test
    void testSquare() {
        assertEquals(25, calc.square(5), "Square of positive number failed");
        assertEquals(25, calc.square(-5), "Square of negative number failed");
        assertEquals(0, calc.square(0), "Square of zero failed");
    }

    @Test
    void testCube() {
        assertEquals(125, calc.cube(5), "Cube of positive number failed");
        assertEquals(-125, calc.cube(-5), "Cube of negative number failed");
        assertEquals(0, calc.cube(0), "Cube of zero failed");
    }

}
```

OUTPUT:

QUESTION – 3:

WRITE NUMBER CLASS HAVING FUNCTION ISPALINDROME(), FACTORIAL(), NUMBER INDIVIDUAL DIGITS SUM. WRITE TEST CASES TO TEST THE FUNCTIONALITY OF NUMBER CLASS.

SOLUTION:

CODE:

```
public class Number {


    public boolean isPalindrome(int n) {

        if (n < 0) {
            return false;
        }

        int original = n;
        int reversed = 0;

        while (n != 0) {
            int digit = n % 10;
            reversed = reversed * 10 + digit;
            n /= 10;
        }

        return original == reversed;
    }

    public long factorial(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("Factorial is not defined for
negative numbers.");
        }

        long fact = 1;
        for (int i = 1; i <= n; i++) {
            fact *= i;
        }
        return fact;
    }


    public int sumOfDigits(int n) {

        n = Math.abs(n);
        int sum = 0;

        while (n > 0) {
            sum += n % 10;
            n /= 10;
```

```
        }

        return sum;
    }
}
```

TEST CLASS:

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class NumberTest {

    Number numberOps = new Number();

    @Test
    void testIsPalindrome() {
        assertTrue(numberOps.isPalindrome(121), "121 should be a palindrome");
        assertTrue(numberOps.isPalindrome(12321), "12321 should be a
palindrome");
        assertTrue(numberOps.isPalindrome(0), "0 should be a palindrome");

        assertFalse(numberOps.isPalindrome(123), "123 is not a palindrome");
        assertFalse(numberOps.isPalindrome(10), "10 is not a palindrome");


        assertFalse(numberOps.isPalindrome(-121), "Negative numbers are not
palindromes");
    }

    @Test
    void testFactorial() {
        assertEquals(120, numberOps.factorial(5), "5! should equal 120");
        assertEquals(720, numberOps.factorial(6), "10! should equal 3628800");


        assertEquals(1, numberOps.factorial(0), "0! should equal 1");
    }

    @Test
    void testFactorialWithNegativeNumbers() {
        assertThrows(IllegalArgumentException.class, () -> {
            numberOps.factorial(-5);
        }, "Factorial of a negative number should throw an exception");
    }

    @Test
    void testSumOfDigits() {
        assertEquals(6, numberOps.sumOfDigits(123), "1+2+3 should equal 6");
        assertEquals(14, numberOps.sumOfDigits(86), "8+6 should equal 14");

        assertEquals(0, numberOps.sumOfDigits(0), "Sum of digits for 0 should
be 0");

        assertEquals(6, numberOps.sumOfDigits(-123), "Sum of digits for -123
```

```
should be 6");
    }
}
```

OUTPUT:

```
✓ NumberTest                20 ms      ✓ 3 tests passed   3 tests total, 20 ms
    ✓ testFactorial()       17 ms      /Library/Java/JavaVirtualMachines/jdk-22.jdk/Contents/Home/bin/java ...
    ✓ testIsPalindrome()     2 ms
    ✓ testSumOfDigits()      1 ms      Process finished with exit code 0
```